

Pandas – Series and DataFrame Basics

Topics Covered

- Introduction to Pandas
- What is a Series?
- Creating a Series
- What is a DataFrame?
- Creating a DataFrame
- Accessing Data
- Real-Time Examples

What is Pandas?

Definition

Pandas is an open-source Python library used for data manipulation, cleaning, analysis, and visualization.

It provides easy-to-use data structures to work with structured data like:

- Excel files
- CSV files
- SQL databases
- JSON files

Why do we use Pandas?

- Handles large datasets efficiently.
- Reads data from different file formats.
- Cleans missing or duplicate values.
- Filters and analyzes data.
- Used in Data Science, AI, Machine Learning, and Business Analytics.

Why is Pandas Important?

The Scenario

Imagine a company has **10 lakh employee records** stored in an Excel sheet.

The Problem with Lists

Using Python lists for these tasks is difficult and time-consuming.

Tasks to Perform

- Find employees with salary > ₹50,000.
- Calculate average salary.
- Remove duplicate records.
- Fill missing values.

The Pandas Solution

Pandas makes these tasks simple with just a few lines of code.

What is a Series?

Definition

A Series is a one-dimensional labeled array.

It contains two main components:

- **Index** (labels)
- **Values** (data)

Think of a Series as a single column in an Excel sheet.

Example

Index	Name
0	Priya
1	Rahul
2	Sneha
3	Kiran

Creating a Series

Default Index

```
import pandas as pd

students =
pd.Series(["Priya",
"Rahul", "Sneha", "Kiran"])

print(students)
```

```
0 Priya
1 Rahul
2 Sneha
3 Kiran
dtype: object
```

Custom Index

```
students = pd.Series(
    ["Priya", "Rahul",
    "Sneha"],
    index=["S101", "S102",
    "S103"]
)
```

```
S101 Priya
S102 Rahul
S103 Sneha
dtype: object
```

Explanation

- **Default Index:** Automatically created (0, 1, 2, 3) when none is specified.
- **Custom Index:** Instead of default numbers, we create meaningful custom labels.
- **Values:** The actual data elements stored in the Series.

Accessing Series Data

Access a Single Value

```
print(students["S101"])
```

Priya

Access Multiple Values

```
print(students[["S101",  
"S103"]])
```

S101	Priya
S103	Sneha

Series Flexibility

Series supports both label-based and positional indexing.

This flexibility makes Series powerful for targeted data retrieval.

What is a DataFrame?

Definition

A DataFrame is a two-dimensional table consisting of rows and columns.

It is similar to:

- Excel Spreadsheet
- SQL Table

Note: Each column is actually a Series.

Example Structure

ID	Name	Age	Marks
101	Priya	22	85
102	Rahul	21	90
103	Sneha	23	88

Creating a DataFrame

```
import pandas as pd

data = {
    "ID": [101, 102, 103],
    "Name": ["Priya", "Rahul", "Sneha"],
    "Age": [22, 21, 23],
    "Marks": [85, 90, 88]
}

df = pd.DataFrame(data)
print(df)
```

Output

	ID	Name	Age	Marks
0	101	Priya	22	85
1	102	Rahul	21	90
2	103	Sneha	23	88

Explanation

- The code creates a dictionary with keys ("ID", "Name", "Age", "Marks") and corresponding lists as values.
- `pd.DataFrame(data)` converts the dictionary into a structured tabular format.
- The output displays a clean table with auto-generated row indices (0, 1, 2).

Accessing Data in a DataFrame

Accessing Columns

```
print(df["Name"])
```

0	Priya	1	Rahul	2	Sneha
---	-------	---	-------	---	-------

Returns a Series containing the data of the specified column.

Accessing Multiple Columns

```
print(df[["Name", "Marks"]])
```

Name	Marks
0 Priya	85
1 Rahul	90

Returns a new DataFrame containing only the selected columns.

Using `iloc()` - Positional

```
print(df.iloc[0])
```

ID 101	Name Priya	Age 22
--------	------------	--------

Uses integer-based positional indexing to access rows by their index number.

DataFrame Properties and Statistics

Shape

```
df.shape
```

```
(3, 4)
```

Returns the dimensions of the DataFrame: 3 rows and 4 columns.

Columns

```
df.columns
```

```
Index(['ID', 'Name', 'Age', 'Marks'],  
      dtype='object')
```

Lists all the column names present in the DataFrame.

Data Types

```
df.dtypes
```

Displays the data type of each column, helping you understand the structure and composition of your data.

Descriptive Statistics

```
df.describe()
```

Provides Count, Mean, Standard Deviation, Minimum, Maximum, and Quartiles. Extremely useful for quick numerical analysis.

Real-Time Example: Student Database

Student Database

ID	Name	Course	Marks
101	Priya	Python	90
102	Rahul	Data Science	85
103	Sneha	AI	95

Using Pandas We Can:

- Find highest marks.
- Find average marks.
- Filter students scoring above 90.
- Export data to Excel.
- Remove duplicate records.

This demonstrates how Pandas transforms raw data into actionable insights.

Series vs. DataFrame & Summary

Comparison

Feature	Series	DataFrame
Dimension	One-dimensional	Two-dimensional
Structure	Single column	Multiple columns
Data Types	One data type	Multiple data types
Access	Has index	Has rows and columns
Use Case	Smaller data	Large datasets

Summary & Activity

- Pandas is one of the most widely used libraries for data analysis in Python.
- A Series stores data in a single labeled column.
- A DataFrame stores data in rows and columns like an Excel spreadsheet.

Classroom Activity:

Create a DataFrame with Name, Age, Marks for 4 students.

```
df = pd.DataFrame(students)
```

Tasks:

1. Display the DataFrame.
2. Print only the Name column.
3. Show the second row using iloc.